Wisdom & Chance TCG

Mobile App AI Agent Sync Guide

Progress Summary & Complete Implementation Reference

Date: March 29, 2026 | Server Version: 2.4.0

Document Purpose

This document provides a comprehensive reference for an AI agent building the Wisdom & Chance TCG mobile app. It covers the current server state, all API endpoints, WebSocket events, data models, authentication, game engine mechanics, and economy/progression systems. The mobile app should consume the existing backend as-is; no server changes are needed.

Server Connectivity

- Production URL: https://wisdom-and-chance.replit.app
- API base path: /api/* (all endpoints)
- WebSocket: wss://wisdom-and-chance.replit.app/ws?token=<jwt>
- Live API Docs (JSON): GET /api/api-docs
- Full Data Dump for Sync: GET /api/admin/sync?code=4838

Table of Contents

1. Project Progress Summary

Completed Features (Tasks 1-7)

- **Task 1 - Combat Mechanic Update:** Guardian & Quick Strike rework. Server-authoritative combat engine resolves all damage, healing, shields, and traits.
- **Task 2 - API Docs & Rules Page:** /api/docs endpoint + /rules page synchronized with 8-step combat resolution flow.
- **Task 3 - Server Config & Feature Flags:** GET /api/config returns feature flags, versioning, and season info for web and mobile.
- **Task 4 - Economy & Collection System:** Gold/Gems/Dust currencies, card rarity (Common/Rare/Epic/Legendary), pack opening, crafting, disenchanting, starter rewards.
- **Task 5 - Mobile Sync Guide & Roadmap:** Comprehensive documentation for mobile development team.
- **Task 6 - Shop & Pack Opening:** 7 pack types, daily deals with discounts, animated pack opening with rarity glow effects.
- **Task 7 - Ranked Seasons & Battle Pass:** 30-day seasons, 6 ranked tiers, 50-level battle pass, weekly challenges, season end rewards, soft rating reset.

Current Server Version: 2.4.0

All 7 tasks are merged and live. The server is fully functional with all systems active. Feature flags control which systems are enabled client-side.

2. Mobile Authentication

Login Flow

- **Endpoint:** POST /api/mobile/auth/login
- **Request Body:** { email: string, firstName?: string, lastName?: string, profileImage?: string, provider?: string }
- **Response:** { token: string, user: { id, email, firstName, lastName, profileImage } }
- **Token Type:**JWT (7-day expiry), signed with server SESSION_SECRET
- **Auto-Provisioning:** New users get: starter decks seeded, 500 gold + starter card collection (2x every Common & Rare card)

Token Refresh

- **Endpoint:** POST /api/mobile/auth/refresh
- **Header:** Authorization: Bearer <current_token>
- **Response:** { token: string } -- fresh 7-day token

Current User

- **Endpoint:** GET /api/mobile/auth/me
- **Header:** Authorization: Bearer <token>
- **Response:** Full user profile object from database

Using the Token

All /api/* endpoints accept the Authorization: Bearer <token> header. The server has a unified auth middleware that works identically for both web sessions and mobile JWT tokens. Every endpoint that works on web will work on mobile with the Bearer token.

JWT Payload: { userId: string, email: string, iat: number, exp: number }

Server maps this to req.user.claims.sub = userId for all endpoint handlers.

3. Complete API Reference

3.1 Player & Profile

- **GET /api/auth/user** -- Get current authenticated user
- **PATCH /api/user/profile**-- Update firstName, lastName, profileImage
- **GET /api/player-stats** -- Game statistics (XP, level, wins/losses, damage dealt)
- **GET /api/player-rating** -- ELO rating and current rank tier
- **GET /api/users/search?q=term** -- Search users by email or name

3.2 Cards & Deck Building

- **GET /api/cards** -- List all cards (id, name, element, power, rarity, trait, buff/debuff)
- **GET /api/cards/:id** -- Single card details
- **GET /api/cards/element/:element** -- Filter by element (fire, water, earth, air, nature)
- **GET /api/commanders** -- List all commander cards with abilities
- **GET /api/commanders/:id** -- Single commander details
- **GET /api/user-decks** -- List saved decks
- **POST /api/user-decks** -- Create deck: { name, commanderId, cardIds[] } validates 40 cards
- **PATCH /api/user-decks/:id**-- Update deck
- **DELETE /api/user-decks/:id** -- Delete deck
- **POST /api/deck-suggestions** -- AI deck suggestion via Gemini (commander + playstyle)
- **GET /api/cards/:id/image** -- Download card artwork (binary image)
- **GET /api/commanders/:id/image** -- Download commander artwork (binary image)

3.3 Economy & Collection

- **GET /api/currencies** -- Player gold, gems, dust balances
- **GET /api/collection** -- Player owned cards with quantities
- **POST /api/collection/starter** -- Grant starter collection (500g + common/rare x2)
- **POST /api/packs/open** -- { packType } Open pack, 5 cards with rarity-weighted pulls
- **POST /api/cards/craft** -- { cardId } Craft card using dust
- **POST /api/cards/disenchant** -- { cardId } Disenchant card for dust

3.4 Shop

- **GET /api/shop/catalog** -- Pack catalog: Standard(100g), Premium(250g), Element(150g)
- **GET /api/shop/daily-deals** -- Daily deal with 15-30% discount, rotates UTC midnight
- **POST /api/shop/purchase** -- { packType } Buy pack with gold
- **POST /api/shop/purchase-bundle** -- { bundleId } Buy bundle with gems

3.5 Social & Friends

- **GET /api/friends** -- List friends with online status
- **DELETE /api/friends** -- { friendId } Remove friend
- **GET /api/friend-requests** -- Pending incoming/outgoing requests
- **POST /api/friend-requests** -- { email } Send friend request
- **POST /api/friend-requests/:id/accept** -- Accept request
- **POST /api/friend-requests/:id/decline** -- Decline request
- **GET /api/friend-messages/:friendId** -- Chat history with friend
- **POST /api/friend-messages/:friendId** -- { message } Send message

3.6 Multiplayer & Rooms

- **GET /api/rooms** -- List public waiting rooms
- **POST /api/rooms** -- { name, isPrivate?, password? } Create room
- **GET /api/rooms/:id** -- Room details (host, guest, spectators)
- **POST /api/rooms/:id/join** -- Join room
- **POST /api/rooms/:id/leave** -- Leave room
- **POST /api/rooms/:id/ready** -- { ready: bool, deckId } Set ready + select deck
- **POST /api/rooms/:id/start** -- Start game (host only, both ready)
- **POST /api/rooms/:id/spectate** -- Join as spectator
- **DELETE /api/rooms/:id/spectate** -- Leave spectating
- **GET /api/rooms/:id/messages** -- Room chat messages
- **POST /api/rooms/:id/messages** -- Send room chat
- **GET /api/leaderboard** -- Top 100 players by ELO rating

3.7 Quests & Achievements

- **GET /api/achievements** -- All possible achievements
- **GET /api/player-achievements** -- Player unlocked achievements
- **POST /api/achievements/:id/claim** -- Claim achievement reward (gold + BP XP)
- **GET /api/daily-challenges** -- Today active challenges
- **GET /api/player-challenges** -- Progress on daily challenges
- **POST /api/player-challenges/:id/claim** -- Claim daily challenge reward

3.8 Ranked Seasons & Battle Pass

- **GET /api/season/current** -- Active season (id, name, number, dates, daysRemaining)
- **GET /api/season/rewards** -- Reward table per tier (gold, packs, dust, cosmetic)
- **GET /api/season/player-rank** -- Player rating, tier, win/loss this season
- **GET /api/season/history** -- Past season records
- **GET /api/battlepass** -- BP progress (level, XP, rewards, claimed status)
- **POST /api/battlepass/claim** -- { level } Claim BP level reward
- **GET /api/weekly-challenges** -- Active weekly challenges with progress
- **POST /api/weekly-challenges/:id/claim** -- Claim completed weekly challenge reward

3.9 Configuration & System

- **GET /api/config** -- Feature flags, server version, active season
- **GET /api/health** -- Server + database health check
- **GET /api/api-docs** -- Full API documentation as JSON

3.10 AI Chat & Image

- **GET /api/conversations** -- List AI chat sessions
- **POST /api/conversations** -- Create new conversation
- **POST /api/conversations/:id/messages** -- Send message, get AI response
- **POST /api/generate-image** -- Generate image from prompt

4. WebSocket Protocol

Connection

Connect: wss://wisdom-and-chance.replit.app/ws?token=<jwt_token>

All messages are JSON: { type: string, payload?: object }

Send "ping" periodically for keepalive. Server responds with "pong".

4.1 Client to Server Events

- **join_room** { roomId } -- Join a game room
- **leave_room** { roomId } -- Leave room
- **join_game** { gameId } -- Join active game session
- **leave_game** { gameId } -- Leave game session
- **room_message** { roomId, message } -- Chat to room
- **game_message** { gameId, message } -- Chat to game
- **game_action** { gameId, action: "draw"|"deploy"|"end_turn"|"use_ability"|"forfeit", data? }
- **player_ready** { roomId, ready: bool, deckId? } -- Toggle ready
- **game_start** { roomId, gameId } -- Start game
- **ping** {} -- Heartbeat

4.2 Server to Client Events

- **auth_success** { userId, displayName } -- On WS connection
- **game_state** SanitizedGameState -- Full game state per player (opponent hand hidden)
- **combat_result** { gameId, ...combatData } -- Combat outcome with damage
- **game_over** { gameId, winnerId, reason } -- Game ended
- **game_error** { gameId, error } -- Action failed
- **user_joined_room** { userId, roomId }
- **user_left_room** { userId, roomId }
- **room_message** { roomId, senderId, senderName, message, timestamp }
- **game_message** { gameId, senderId, senderName, message, timestamp }
- **player_ready** { playerId, ready, deckId? }
- **game_start** { gameId } -- Game began
- **presence_update** { userId, status: "online"|"offline" }
- **opponent_disconnected** { gameId, disconnectedPlayerId, reconnectTimeout: 60 }
- **opponent_reconnected** { gameId, reconnectedPlayerId }
- **error** { message } -- General error
- **pong** {} -- Heartbeat response

5. Game Engine & Combat Mechanics

Server-Authoritative Model

The server is the single source of truth. Clients send actions, the server validates them, updates state, and broadcasts sanitized state back. NEVER compute game logic locally for PvP. The opponent hand and face-down cards are hidden from each player.

Game Phases (per turn)

- **1. Draw Phase:** Both players draw cards simultaneously from their deck.
- **2. Deployment Phase:** Players select cards from hand to place face-down on battlefield.
- **3. Combat Phase:** Cards revealed, power calculated with buffs/debuffs/traits, damage dealt.
- **4. Cleanup:** Battlefield cleared, cards to Yard. Turn increments, back to Draw.

Combat Resolution (8 Steps)

- Step 1: Reveal all face-down cards
- Step 2: Calculate base power for each card
- Step 3: Apply buff modifiers from friendly cards
- Step 4: Apply debuff modifiers from enemy cards
- Step 5: Apply commander ability effects (shield, first_strike, heal, group buffs)
- Step 6: Calculate total power per side, determine winner
- Step 7: Apply traits: Quick Strike (pre-combat dmg), Guardian (blocks), Restoration (heals)
- Step 8: Apply base damage (power difference) to loser HP

Victory Conditions

- Reduce opponent HP to 0 (Starting HP: 40)
- Opponent forfeits
- Opponent disconnects for 60+ seconds

Commander Abilities

4 categories: (1) Group Buffs/Debuffs on matching element; (2) Trait Activation (first_strike/shield/heal); (3) Trait-Like Group Effects; (4) Extra Deployments. Cost Victory Points or Withdrawal Points to use.

6. Key Data Models

6.1 Card

{ id, name, element (fire|water|earth|air|nature), power (1-10), trait (quick_strike|care_package|restoration|guardian|null), traitValue, buffElement, buffAmount, debuffElement, debuffAmount, isCommander }

Rarity derived from power: Common (1-3), Rare (4-6), Epic (7-8), Legendary (9-10).

6.2 Commander

{ id, name, element, abilities: [{ name, description, phase, costType (victory|withdrawal), costAmount, effectType, effectValue, targetElement }] }

6.3 Deck

{ id, userId, name, commanderId, cardIds: string[] }. Rules: 40 cards, max 3 copies, 1 commander. Server validates.

6.4 Currencies

{ userId, gold, gems, dust }. Starter: 500g/0gems/0dust. Match: Win=30g, Loss=10g, Draw=15g.

6.5 Collection

{ userId, cardId, quantity }. Starter: 2x every Common+Rare card (power 1-5).

6.6 Player Rating

{ userId, rating (ELO, starts 1000), highestRating, wins, losses, streak }

Tiers: Bronze(0+), Silver(1100+), Gold(1300+), Platinum(1500+), Diamond(1700+), Master(2000+).

6.7 Season

{ id, name, seasonNumber, startsAt, endsAt, isActive }. 30-day default. Auto-transitions.

6.8 Battle Pass

50 levels/season. XP = level*200 (cumulative). Sources: Win(+100), Loss(+30), Daily(+150), Weekly(+300), Achievement(+200). Rewards: gold, gems, dust, packs, specific cards.

6.9 Weekly Challenges

3/week/season. Types: win_games, play_element, deal_damage. Grants gold + BP XP.

7. Economy Constants & Pack Types

Pack Types

- **Standard:** 100 gold, 5 cards. Common 60%, Rare 25%, Epic 10%, Legendary 5%
- **Premium:** 250 gold, 5 cards. Guaranteed 1+ Rare or better
- **Element Packs:** 150 gold each (Fire/Water/Earth/Air/Nature), filtered to element

Crafting/Disenchant (Dust)

- Common: 40 craft / 5 disenchant
- Rare: 100 craft / 20 disenchant
- Epic: 400 craft / 100 disenchant
- Legendary: 1600 craft / 400 disenchant

Season End Rewards (by Peak Tier)

- Bronze: 100g, 1 pack, 50 dust
- Silver: 200g, 2 packs, 100 dust, silver_card_back cosmetic
- Gold: 400g, 3 packs, 200 dust, gold_card_back cosmetic
- Platinum: 600g, 5 packs, 400 dust, platinum_card_back cosmetic
- Diamond: 1000g, 8 packs, 600 dust, diamond_card_back cosmetic
- Master: 1500g, 12 packs, 1000 dust, master_card_back cosmetic

Feature Flags (GET /api/config)

Fetch on startup, show/hide UI accordingly:

- economy_enabled -- currencies, collection, crafting
- shop_enabled -- shop catalog, purchases, daily deals
- multiplayer -- rooms, matchmaking, PvP
- ranked_seasons -- season display, ranking
- battle_pass -- BP progress, claims
- weekly_challenges -- weekly quest system
- achievements -- achievement tracking
- daily_challenges -- daily quest system

8. Mobile App Implementation Checklist

Phase 1: Core Foundation

- Auth screen with email login -> POST /api/mobile/auth/login
- Secure JWT storage (iOS Keychain / Android EncryptedSharedPreferences)
- Auto-refresh token before expiry (POST /api/mobile/auth/refresh)
- Fetch + cache all cards/commanders (GET /api/cards, GET /api/commanders)
- Download card artwork (GET /api/cards/:id/image), cache locally
- Fetch feature flags on startup (GET /api/config)

Phase 2: Collection & Deck Building

- Collection screen: GET /api/collection merged with card data
- Deck builder: 40-card validation, max 3 copies, commander selection
- Deck CRUD: GET/POST/PATCH/DELETE /api/user-decks
- Card detail view: stats, element, rarity badge, artwork

Phase 3: Shop & Economy

- Currency bar: GET /api/currencies (gold/gems/dust)
- Shop screen: GET /api/shop/catalog
- Daily deals with countdown: GET /api/shop/daily-deals
- Purchase flow: POST /api/shop/purchase
- Pack opening animation: 5 card reveals with rarity effects
- Crafting/disenchanting UI: POST /api/cards/craft, /disenchant

Phase 4: Multiplayer

- WebSocket via wss://...?token=<jwt>
- Room browser (GET /api/rooms) + create room
- Pre-game lobby: ready system + deck selection
- Game board: render all phases (draw/deploy/combat/cleanup)
- Display server-sent sanitized game state (never compute locally)
- Combat animation, in-game chat, spectator mode
- Disconnect handling: 60s reconnect window

Phase 5: Social

- Friends list with online status
- Friend request send/accept/decline
- Direct messaging
- Leaderboard: GET /api/leaderboard

Phase 6: Progression

- Season info: GET /api/season/current
- Player rank with tier badge: GET /api/season/player-rank
- Season history: GET /api/season/history
- Battle pass: 50 levels, progress bar, claims (GET /api/battlepass)
- Weekly challenges with progress (GET /api/weekly-challenges)
- Daily challenges + achievements with claim flows
- Player stats: GET /api/player-stats

Phase 7: Polish

- Push notifications: friend requests, challenge completions, season transitions
- Offline mode with cached card data
- Profile editing: PATCH /api/user/profile
- App icon/splash screen, dark theme matching web app
- Loading/skeleton states for all data fetches

9. Critical Implementation Notes

- **Server-Authoritative:** NEVER compute game logic on mobile for PvP. Send actions via WebSocket, render what server sends back.
- **Unified Auth:** Every /api/ endpoint works with Bearer token. No separate mobile endpoints needed except /api/mobile/auth/*.
- **Element System:** 5 elements: fire, water, earth, air, nature. Cards have buff/debuff modifiers targeting specific elements.
- **Card Rarity:** Derived from power: Common(1-3), Rare(4-6), Epic(7-8), Legendary(9-10). Calculated, not stored.
- **Deck Rules:** Exactly 40 cards, max 3 copies, 1 commander. Server validates on save.
- **Starting HP:** 40 HP per player. Guardian blocks damage. Restoration heals.
- **ELO Rating:** Starts 1000. Updated after ranked matches. Soft reset at season end toward 1000.
- **Admin Email:** redeagle28089@gmail.com -- only this user has admin access.

10. Key Server File Reference

Server-side files for reference (mobile app does NOT modify these):

- **shared/schema.ts** -- Core game types (Card, Commander, Deck, Game, GameState)
- **shared/models/economy.ts** -- Economy DB tables + constants (currencies, packs, seasons, BP)
- **shared/models/multiplayer.ts** -- Social DB tables (friends, rooms, ratings, achievements)
- **shared/models/auth.ts** -- User, session, deck, card image tables
- **server/routes.ts** -- All REST API endpoints (~2800 lines)
- **server/multiplayerRoutes.ts** -- Social, room, leaderboard endpoints
- **server/websocket.ts** -- WebSocket server: game actions, chat, presence
- **server/gameEngine.ts** -- Server-authoritative combat engine
- **server/economyService.ts** -- Helpers: grantGold, grantBattlePassXP, openPackForUser
- **server/seasonService.ts** -- Season transition checker, rewards distribution
- **server/mobileAuth.ts** -- Mobile JWT auth endpoints
- **server/unifiedAuth.ts** -- Unified auth middleware (session + JWT)
- **server/apiDocs.ts** -- API documentation generator

This document was generated on March 29, 2026 for server version 2.4.0.

For the most current API reference, check GET /api/api-docs or GET /api/admin/sync?code=4838.

The mobile app should be a pure consumer of these APIs. No server modifications required.